CO - 3 ASSESSMENT TOOL -1

Name : Lanka Kavya **Subject Code : CSA0619**
Registration Number : 192472298 **Subject Name : DAA**

Q17 : **Analyze the effect of input ordering on Merge Sort's merging phase by measuring merge operation counts for different datasets. Record execution time and merge complexity. Interpret why Merge Sort remains unaffected by input patterns. Justify its predictable performance.**

Sol :

Source code :

```
import time

count = 0

def merge(a, left, mid, right):

    global count

    L = a[left:mid + 1]

    R = a[mid + 1:right + 1]

    i = j = 0

    k = left

    while i < len(L) and j < len(R):

        count += 1

        if L[i] <= R[j]:

            a[k] = L[i]

            i += 1

        else:

            a[k] = R[j]

            j += 1
```

```
        k += 1
    while i < len(L):
        a[k] = L[i]
        i += 1
        k += 1
    while j < len(R):
        a[k] = R[j]
        j += 1
        k += 1
def merge_sort(a, left, right):
    if left < right:
        mid = (left + right) // 2
        merge_sort(a, left, mid)
        merge_sort(a, mid + 1, right)
        merge(a, left, mid, right)
datasets = {
    "Sorted": [1, 2, 3, 4, 5, 6, 7, 8],
    "Reverse": [8, 7, 6, 5, 4, 3, 2, 1],
    "Random": [4, 1, 7, 3, 8, 2, 6, 5]
}
for name, data in datasets.items():
    count = 0
    start = time.time()
    merge_sort(data, 0, len(data) - 1)
    end = time.time()
```

```
print(name, "-> Merge Comparisons:", count,
      "Time:", end - start)
```

Aim

To analyze how sorted, reverse-sorted, and random inputs affect the merging phase of Merge Sort.

Objective

- Count merge comparisons.
- Measure execution time.
- Compare different input patterns.
- Justify Merge Sort's predictable performance.

Datasets

- **Sorted:** Elements already in ascending order.
- **Reverse:** Elements arranged in descending order.
- **Random:** Elements in random order.

Observation

Input	Merge Comparisons	Performance
Sorted	Low/Moderate	Fast
Reverse	Low/Moderate	Fast
Random	Moderate/High	Slightly higher

The exact comparison count depends on the input values, but the number of **splitting and merging levels remains the same**.

Merge Complexity

At every level, Merge Sort processes approximately n elements, and there are $\log n$ levels.

O(nlogf_0n)O(n\log n)

Therefore:

- Best case: **O(n log n)**
- Average case: **O(n log n)**
- Worst case: **O(n log n)**

Why Input Pattern Has Little Effect

Merge Sort always divides the array into smaller parts and merges them systematically. Unlike algorithms such as Quick Sort, it does not depend heavily on selecting a good pivot based on input ordering.

Therefore, sorted, reverse, and random inputs all maintain the same overall complexity.

Result

Merge Sort provides **predictable performance of O(n log n)** regardless of input ordering. Input patterns may slightly change the number of comparisons and execution time, but they do not change the overall complexity.

Output Screenshot :